PONTIFICIA UNIVERSIDAD CATOLICA MADRE Y MAESTRA

PUCMM

Nombres:

Luis José Paulino ID: 10160913

Jadiel Garcia ID: 10161146

Asignatura:

Algoritmos clásicos y estructuras de datos

Maestro:

Freddy Peña

Tema:

Reporte proyecto final Sistema Integrado de Rutas

Abril 2026

REQUERIMIENTOS OBLIGATORIOS DEL ANALISIS

1. Operaciones sobre la estructura del grafo

a. Inserción de una parada.

La inserción de una parada en la clase Grafo utilizando un HashMap tiene una complejidad de O(1) (en el caso promedio).

Debido a que el método anadirParada realiza una verificación de existencia con containsKey() seguida de una inserción con put(). En un HashMap, ambas operaciones toman tiempo constante y no dependen del número actual de paradas |V| ni de las rutas |E|.

b. Inserción de una ruta.

La inserción de una ruta utilizando el método anadirRuta en la clase Grafo tiene una complejidad de O(1) (en el caso promedio).

Debido a que el método realiza búsquedas en el HashMap (containsKey) para validar el origen y el destino (O(1)), inserta la ruta en la lista interna de la parada origen O(1), y finalmente inserta la ruta en el mapa global de rutas mediante put() (O(1)).

c. Eliminación de una ruta.

La eliminación de una ruta utilizando el método eliminarRuta(Ruta ruta) tiene una complejidad de O(E) (donde E representa el grado de salida o número de rutas de la parada origen).

Debido a que, aunque eliminar del HashMap global rutas toma O(1), el llamado a ruta.getParadaOrigen().eliminarRuta(ruta) implica buscar y remover un elemento específico dentro de una estructura de lista (List), lo cual toma un tiempo lineal proporcional a la cantidad de rutas conectadas a esa parada específica.

d. Consulta de vecinos de una parada.

La consulta de los vecinos (destinos accesibles) desde una parada tiene una complejidad de O(E) (donde E es el número de rutas salientes de esa parada en particular).

Debido a que, aunque acceder a la lista de adyacencia interna (getRutasSalientes()) es O(1), para obtener o consultar específicamente a los "vecinos", el sistema debe iterar a través de esta lista para recuperar la propiedad paradaDestino de cada arista conectada.

e. Verificación de existencia de una arista.

La verificación de existencia de una arista utilizando el método obtenerRutaDirecta(Parada p1, Parada p2) tiene una complejidad de O(E) (donde E es el número total de rutas en todo el grafo).

Debido a que la implementación actual realiza un ciclo for que itera secuencialmente sobre rutas.values(), evaluando cada ruta del sistema hasta encontrar una coincidencia exacta de origen y destino.

2. Algoritmos Implementados

a. BFS

La implementación de BFS tiene una complejidad de O(V*E).

Debido a que, por cada vértice extraído de la cola, el algoritmo itera sobre todas las rutas globales del grafo (grafo.getRutas().values()) buscando coincidencias con el origen, en lugar de iterar únicamente sobre las listas de adyacencia del vértice actual.

b. DFS

La implementación de DFS tiene una complejidad de O(V*E).

Debido a que, en cada llamada recursiva, el algoritmo vuelve a recorrer el mapa completo de rutas globales (grafo.getRutas().values()) buscando las aristas salientes del nodo actual, en lugar de acceder directamente a sus vecinos.

c. Dijkstra

La implementación de Dijkstra tiene una complejidad de O(V^2 + V*E).

Debido a que no utiliza una cola de prioridad (PriorityQueue). En su lugar, realiza una búsqueda lineal iterando sobre el mapa distancia.keySet() para encontrar el vértice no visitado con la distancia mínima. Además, para la relajación de aristas, itera sobre todas las rutas globales (grafo.getRutas().values()) por cada vértice, lo que añade O(V*E).

d. Bellman-Ford

La implementación de Bellman-Ford tiene una complejidad de O(V*E).

Debido a que el algoritmo principal consta de un bucle que se ejecuta V - 1 veces, y dentro de este bucle itera sobre todas las aristas globales E (grafo.getRutas().values()) para relajar los pesos. La comprobación final de ciclos negativos añade otra iteración sobre E, manteniendo el límite asintótico en O(V*E).

e. Floyd-Warshall

La implementación de Floyd-Warshall tiene una complejidad de O(V^3).

Debido a que utiliza tres bucles anidados (k, i, j), cada uno iterando desde 0 hasta el número total de vértices V, para calcular los caminos más cortos entre todos los pares de nodos posibles. La fase inicial de llenar la matriz y la reconstrucción del camino toman O(E) y O(V) respectivamente, siendo dominadas por los bucles anidados cúbicos.

ESTRUCTURA PRINCIPAL

El proyecto fue realizado utilizando una clase grafo, el mismo contiene un mapa para todas las paradas, y otro para todas las rutas, en base a este se encuentra el funcionamiento de todo el programa.

```
public class Grafo {  LuijoGift +2

    // MAPA PRINCIPAL DEL GRAFO.
    // GUARDA CADA PARADA POR SU ID PARA ACCESO RÁPIDO.
    private Map<String, Parada> paradas;  11 usages
    private Map<String, Ruta> rutas;  9 usages
```

La clase ruta posee las propiedades de una arista, además de un ID para su correcta identificación, y atributos tipo parada de origen y destino, representando desde donde y hasta donde salen las rutas.

```
public class Ruta {  Jadiel García Núñez +2

    private String id;  3 usages
    private Parada paradaOrigen;  2 usages
    private Parada paradaDestino;  2 usages
    private double tiempoViaje;  3 usages
    private double distancia;  3 usages
    private double costo;  3 usages
    private int transbordos;  3 usages
```

Por otro lado, las paradas poseen un ID, NOMBRE, una DESCRIPCION, ZONA y una lista de las rutas que salen de esta parada.

```
public class Parada {  Jadiel García Núñez +2

    private String id;  2 usages
    private String nombre;  4 usages
    private double x;  3 usages
    private double y;  3 usages
    private boolean activa;  3 usages
    private String descripcion;  2 usages
    private String zona;  2 usages
    private List<Ruta> rutasSalientes;  4 usages
```

ALGORITMOS IMPLEMENTADOS

Para la realización del programa los algoritmos que se utilizaron para conocer el camino de una parada a otra fueron:

- Dijkstra (Principal)
- BellmanFord
- FloydWarshall
- BFS
- DFS

DECISIONES DE DISEÑOS

Se utilizó un ENUM para el criterio de optimización, este básicamente es el que se toma en cuenta al momento de calcular la mejor ruta, tomando en cuenta tiempo, distancia, costo o transbordo.

Se utiliza una base de datos en PostgreSQL conectada a través de Docker para la persistencia de datos.

Se utiliza la librería SmartGraph de Bruno Silva para la visualización del mapa del grafo.

El programa trabaja sobre Dijkstra, para la implementación de los demás algoritmos está un pequeño comboBox, que al seleccionarlo se activan las opciones avanzadas, donde el usuario selecciona el algoritmo de su preferencia.

Los controladores trabajan sobre la misma instancia de HellowController para mantener todos los datos trabajando sobre la misma base.

Se colocó un ToolTip para que al pasar el cursor por encima de una ruta o parada se pueda visualizar la información de este.

Para visualizar la ruta que se ha creado, hay un cuadro en la esquina superior derecha que muestra desde donde y hasta donde, el costo y por todas las pardas que se debe pasar para llegar al destino.

Decisiones tomadas para grafo conexo:

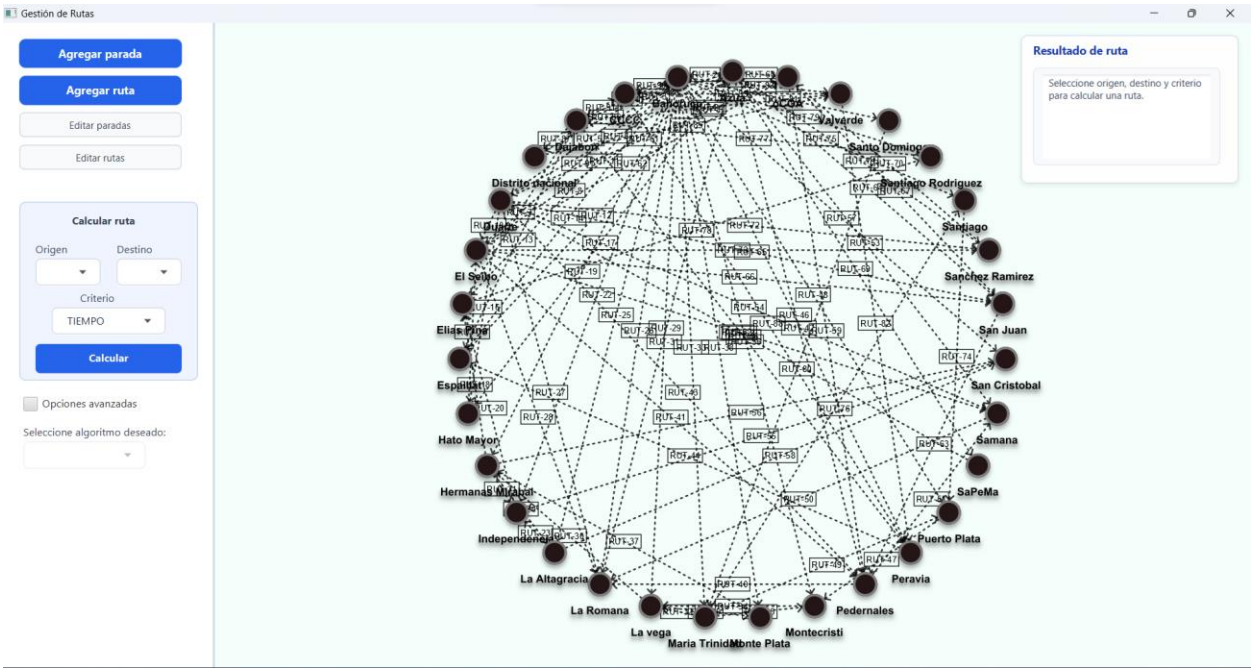
Al crear una parada es obligatorio crear una ruta que salga de esta, tomar en cuenta:

- En caso de que sea la primera parada creada no se obligará al usuario a crear una ruta.
- Para inicializar el grafo siendo conexo, al crear la segunda parada y esta obligue a crear una ruta a la primera, la primera creará una ruta exactamente igual hacia la segundo, a partir de aquí el grafo inicia siendo conexo.

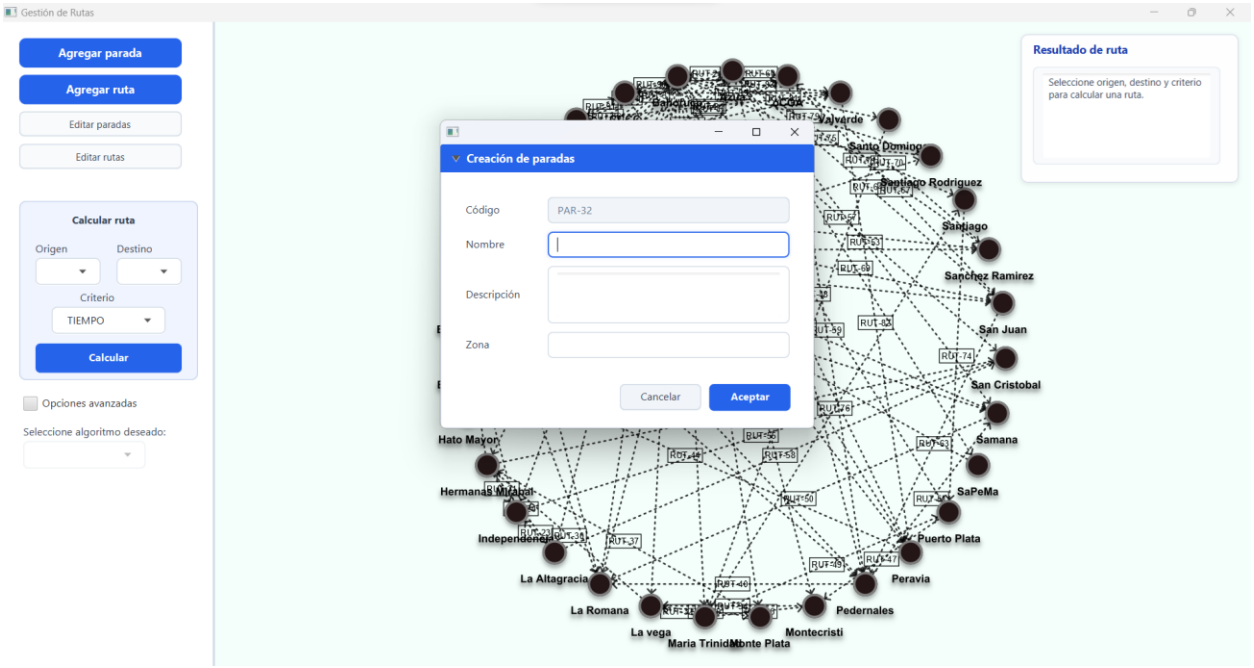
Al crear una parada A y se obliga a crear una ruta hacia una parada B, para que el grafo sea conexo, todas las rutas que tengan una ruta hacia la parada B, también tendrán una generada de forma automática hacia la parada A, esta se crea tomando en cuenta la suma de las propiedades de la ruta de A->B y de cualquier parada->B.

Para que el usuario este consciente de sus cambios, se mostrará una advertencia antes de eliminar una parada o ruta notificando que esto podría hacer que el grafo deje de ser conexo.

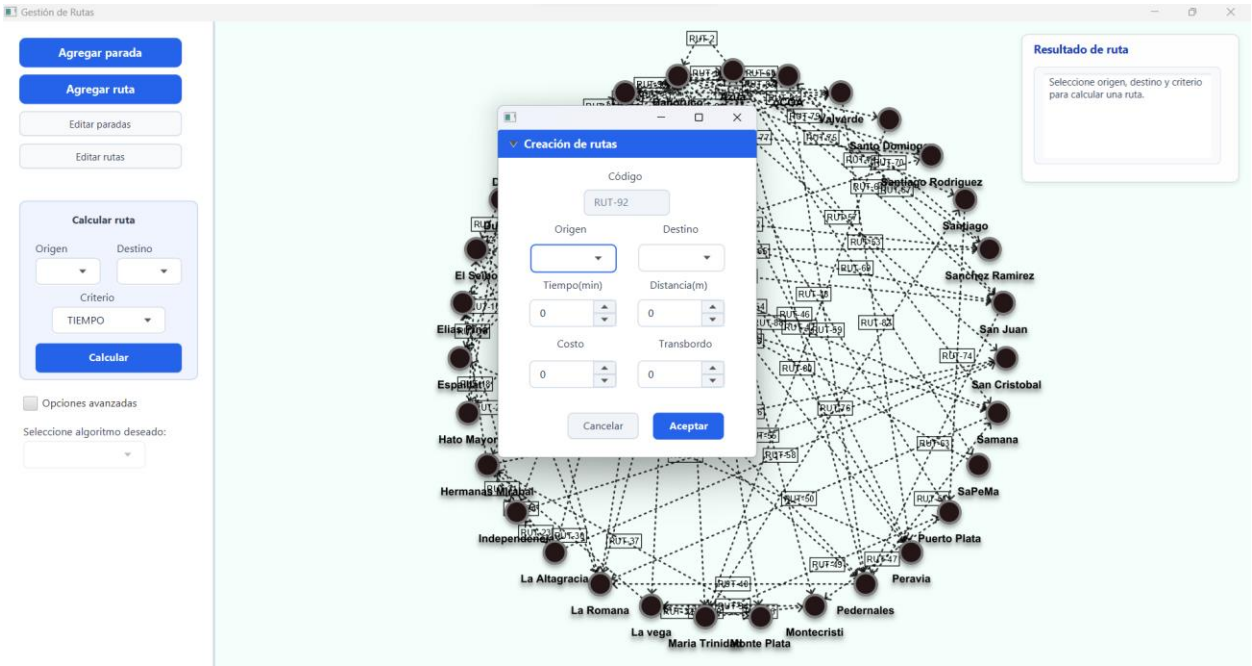
Pruebas realizadas sobre grafo con 31 paradas:



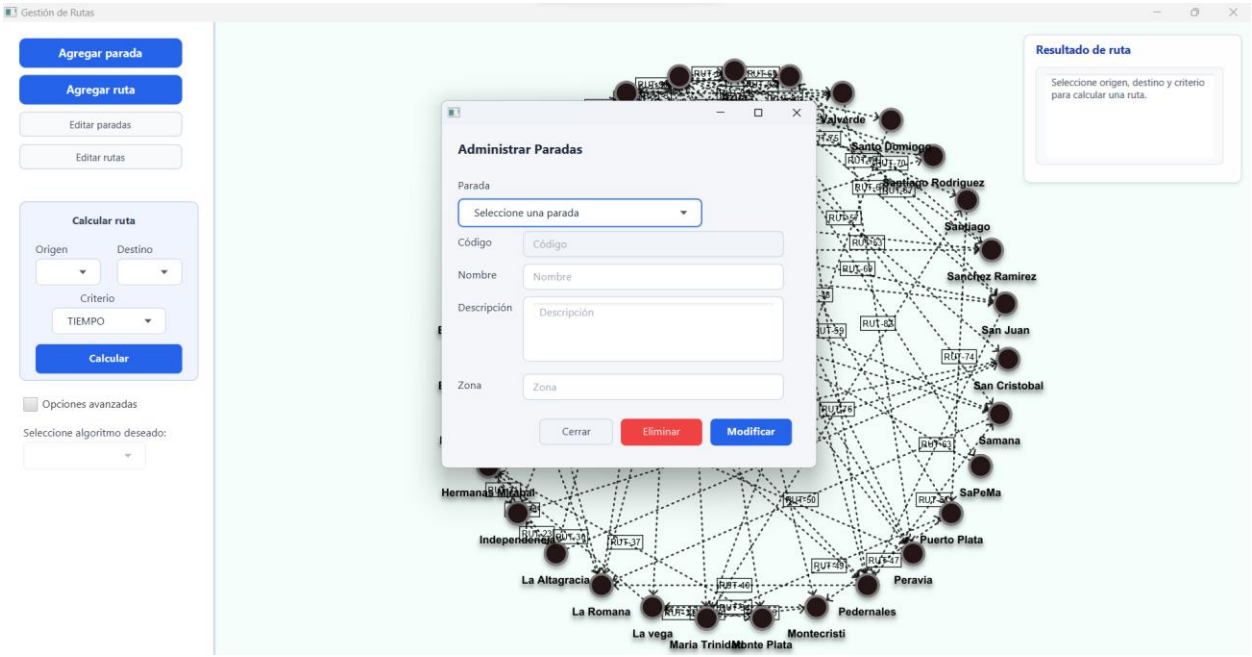
Menú para agregar paradas:



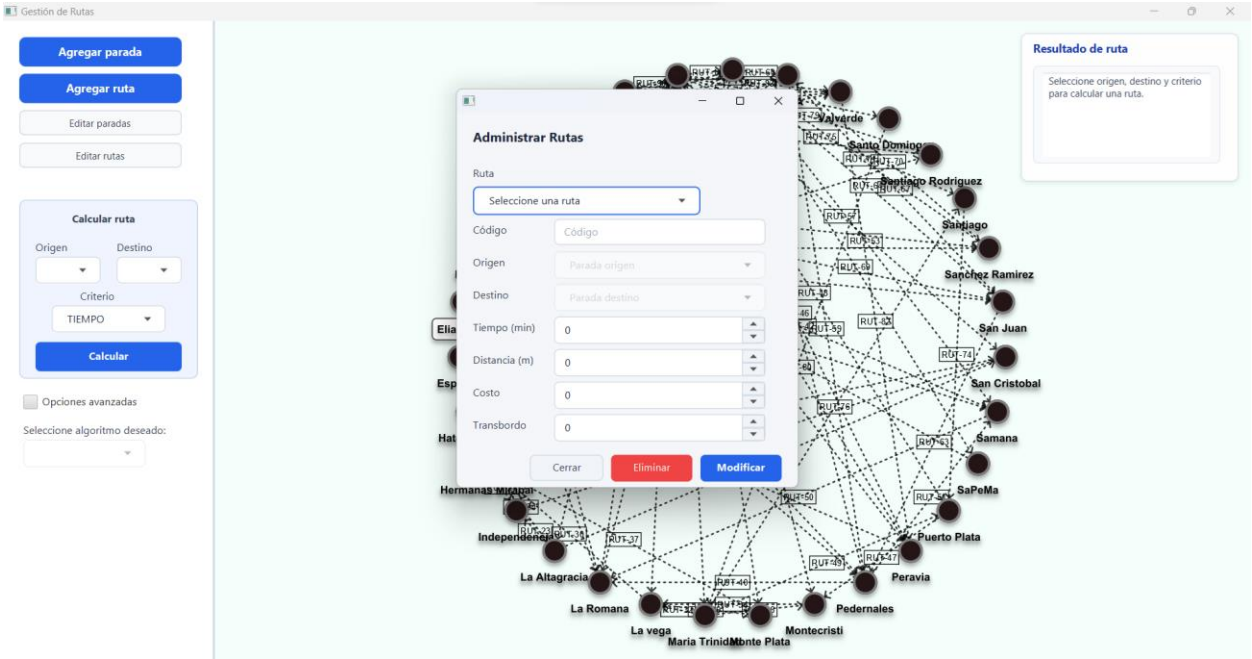
Menú para creación de rutas:



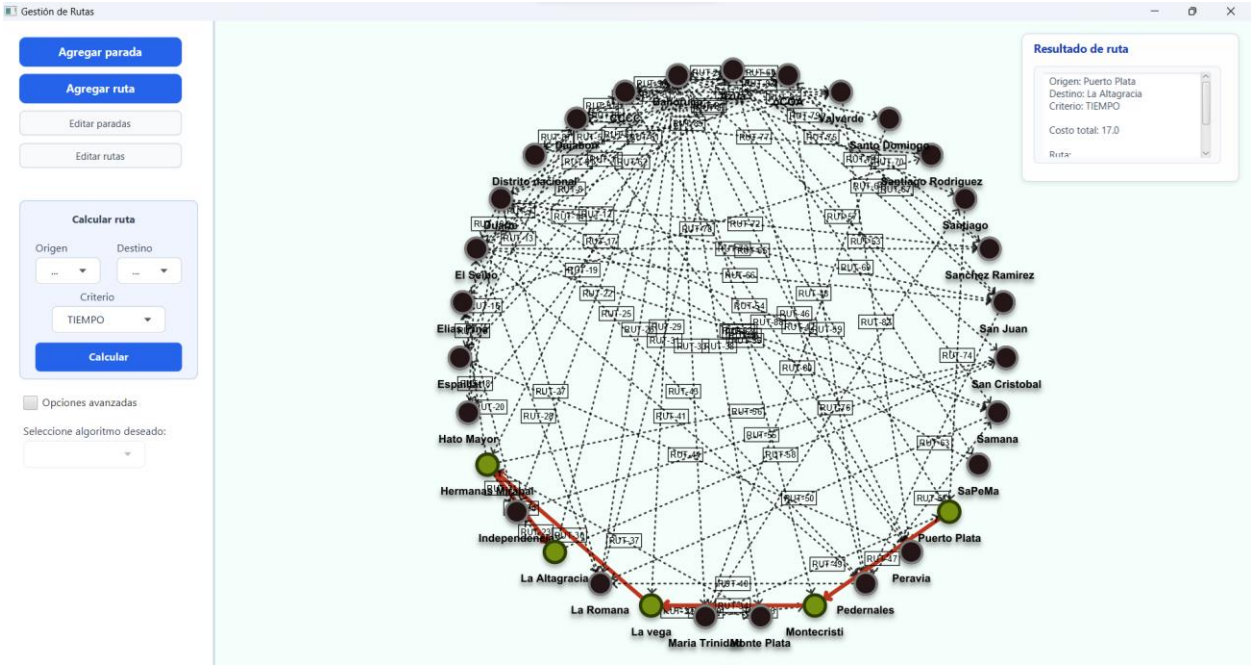
Menú edición y eliminación de paradas



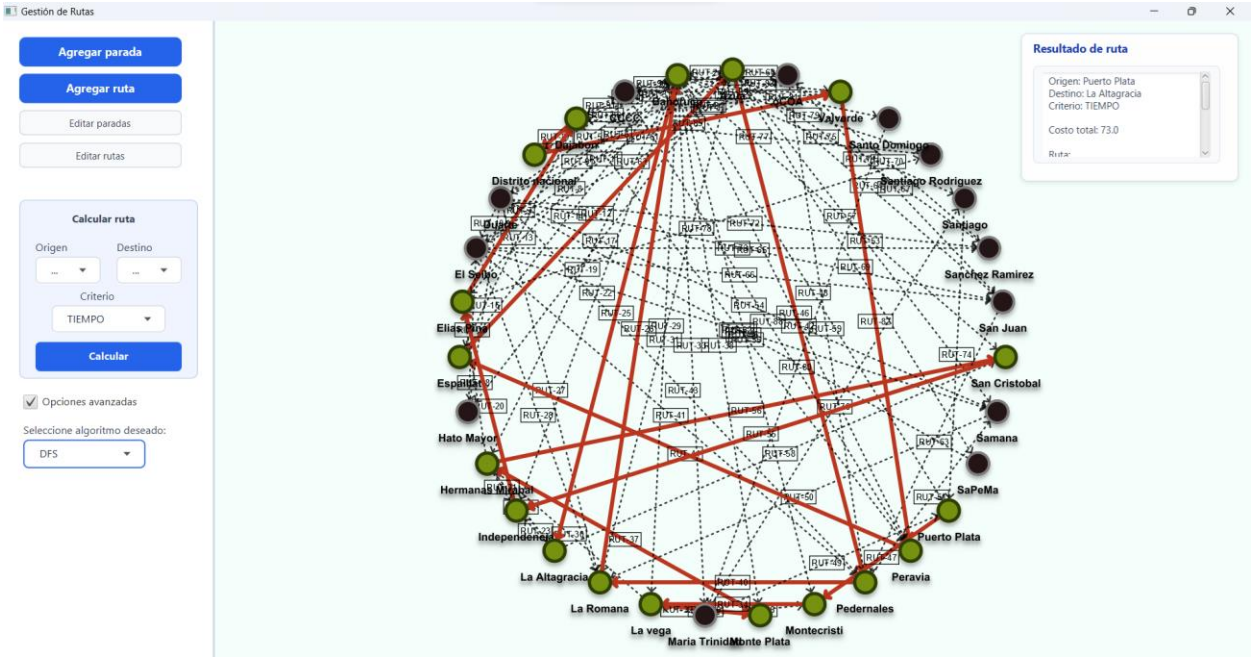
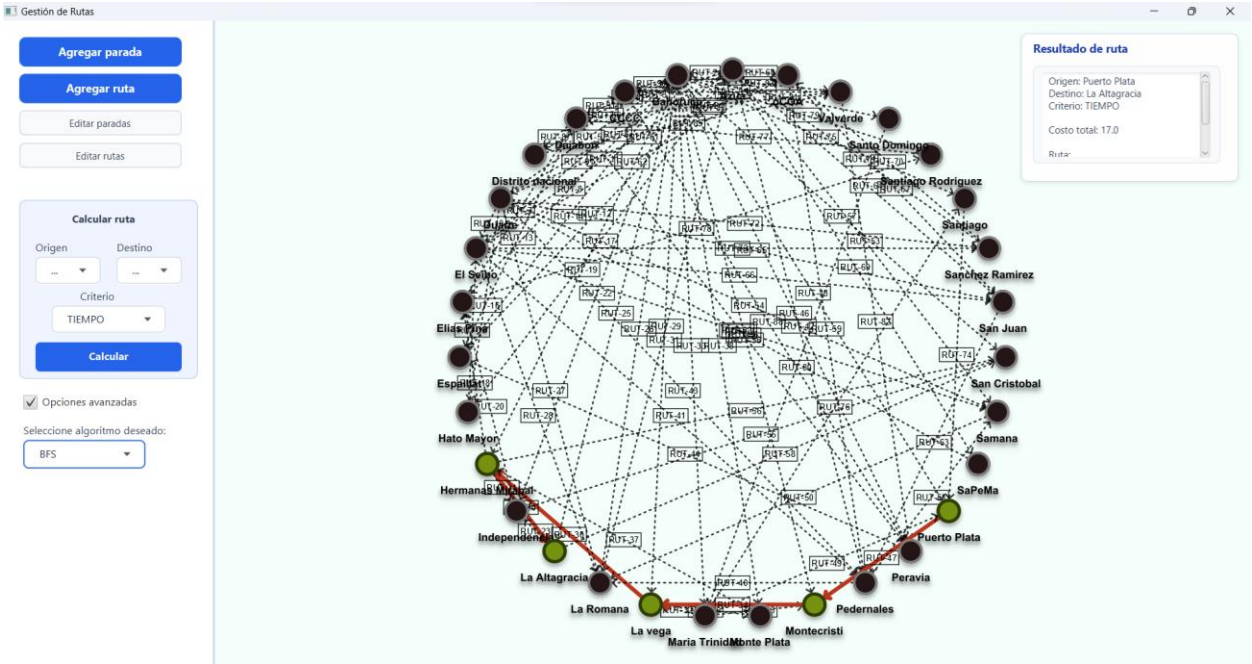
Menú para modificación y eliminación de rutas



Cálculo de Puerto Plata -> Altagracia (utilizando Dijkstra con criterio de tiempo:



Mismo requerimiento utilizando DFS:



Mismo requerimiento utilizando BFS: